

Atividade 01 — Busca linear x Busca binária

ICD-004 - Análise de Algoritmos

Aula 03 - Atividade 01

Aluno: **Wellington Junior Torres de Melo**

Data: 18/08/2026

1. O que foi testado

Foram implementados dois algoritmos de busca em Python e o tempo de cada um foi medido com `time.perf_counter()`. O teste usa uma lista grande, com **1.000.000 de números ordenados** (de 0 a 999.999), e procura o **último número da lista** — que é o pior caso para a busca linear, porque ela precisa passar por todos os elementos antes de achar. O teste foi repetido **3 vezes** para mostrar que o resultado não foi sorte de uma execução.

2. Resultado por tempo de cada execução

Em cada execução aparecem dois resultados: o **1º resultado** é o tempo da busca linear e o **2º resultado** é o tempo da busca binária procurando exatamente o mesmo número, na mesma lista.

Execução	1º resultado Busca linear - O(n)	2º resultado Busca binária - O(log n)	Quantas vezes mais rápida
1	184797.3439 ms	1.4966 ms	123478x
2	207764.4876 ms	0.6959 ms	298555x
3	195113.0113 ms	0.3106 ms	628181x
Média	195891.6143 ms	0.8344 ms	234779x

Conclusão do teste: a busca binária foi cerca de **234779 vezes mais rápida** que a busca linear (média de 0.8344 ms contra 195891.6143 ms). O motivo aparece na contagem de passos: a busca linear fez **1.000.000 de comparações** e a busca binária fez apenas **20**, porque cada passo dela joga fora metade da lista.

3. Saída completa do programa

```
COMPARACAO POR TEMPO: BUSCA LINEAR x BUSCA BINARIA
=====
Tamanho da lista ...: 1000000000 numeros (de 0 ate 999999999)
Numero procurado ...: 999999999 (o ultimo da lista = pior caso)
Execucoes .....: 3

EXECUCAO 1
1o resultado - busca linear : posicao 999999999 | 1000000000 passos | 184797.3439 ms
2o resultado - busca binaria : posicao 999999999 | 30 passos | 1.4966 ms
--> a binaria foi 123478 vezes mais rapida

EXECUCAO 2
1o resultado - busca linear : posicao 999999999 | 1000000000 passos | 207764.4876 ms
2o resultado - busca binaria : posicao 999999999 | 30 passos | 0.6959 ms
--> a binaria foi 298555 vezes mais rapida

EXECUCAO 3
1o resultado - busca linear : posicao 999999999 | 1000000000 passos | 195113.0113 ms
2o resultado - busca binaria : posicao 999999999 | 30 passos | 0.3106 ms
--> a binaria foi 628181 vezes mais rapida

MEDIA DAS 3 EXECUCOES
=====
Busca linear - O(n) .....: 195891.6143 ms
```

```
Busca binaria - O(log n) .: 0.8344 ms
A busca binaria foi 234779 vezes mais rapida.
```

```
PASSOS NO PIOR CASO (o que explica a diferenca de tempo)
=====
Busca linear : 1000000000 passos (olhou a lista toda)
Busca binaria : 30 passos (cortou a lista no meio 30 vezes)
```

4. Respostas das questões

a) Qual a diferença entre os dois algoritmos em termos de quantidade de passos no pior caso?

No pior caso a busca linear precisa olhar todos os elementos, ou seja, **n passos** (no teste: 1.000.000). A busca binária corta a lista no meio a cada passo, então ela precisa de **log2(n) passos** (no teste: 20). A diferença mais importante é o que acontece quando a lista cresce: se eu dobrar o tamanho da lista, a busca linear passa a fazer o dobro de passos, enquanto a busca binária faz apenas **1 passo a mais**.

b) Por que a busca binária exige que a lista esteja ordenada?

Porque ela compara o alvo com o elemento do meio para decidir **qual metade jogar fora**. Quando ela conclui “o alvo é maior que o meio, então ele está na direita”, ela está afirmando algo sobre elementos que nem olhou — e isso só é verdade se a lista estiver ordenada. Em uma lista desordenada ela jogaria fora a metade que continha o alvo e diria que o número não existe. Ou seja, sem ordenação ela não fica lenta: ela fica **errada**.

c) Escreva em suas palavras a diferença entre O(n) e O(log n).

O(n) e O(log n) não são algoritmos: são formas de dizer **como o trabalho cresce** quando a entrada cresce. Em **O(n)** o trabalho cresce junto com a lista — lista 10 vezes maior, 10 vezes mais trabalho; é o caso da busca linear, que pode ter que olhar tudo. Em **O(log n)** cada passo elimina metade do que sobrou, então o trabalho cresce muito devagar — lista 1.000 vezes maior custa apenas cerca de 10 passos a mais; é o caso da busca binária. Em uma lista de 1 bilhão de números, o pior caso de O(n) são 1 bilhão de comparações e o de O(log n) são 30 comparações.

5. Script usado no teste

Arquivo **busca_linear_vs_binaria.py** — é este script que faz as buscas e mede os tempos mostrados na seção 2.

```
"""
ICD-004 - Analise de Algoritmos
Aula 03 - Atividade 01

Compara a BUSCA LINEAR (O(n)) com a BUSCA BINARIA (O(log n)),
medindo o TEMPO que cada uma leva em varias execucoes.

Para rodar: python busca_linear_vs_binaria.py
"""

import time
from pathlib import Path

# Pasta onde este arquivo esta (para salvar os resultados aqui do lado).
PASTA = Path(__file__).parent

# -----
# Configuracao do teste
# -----
TAMANHO = 1000000000 # lista grande: 1 milhao de numeros
EXECUCOES = 3 # quantas vezes vamos repetir o teste

# -----
# 1) BUSCA LINEAR - O(n)
# Olha um elemento por vez, do inicio ao fim, ate achar.
# Nao precisa de lista ordenada.
# -----
def busca_linear(lista, alvo):
    passos = 0
    for posicao in range(len(lista)):
        passos = passos + 1 # contou mais uma comparacao
        if lista[posicao] == alvo:
            return posicao, passos # achou!
    return -1, passos # nao achou

# -----
# 2) BUSCA BINARIA - O(log n)
# Olha o elemento do MEIO e joga fora a metade que nao pode conter o alvo.
# So funciona se a lista estiver ORDENADA.
# -----
def busca_binaria(lista, alvo):
    passos = 0
    inicio = 0
    fim = len(lista) - 1

    while inicio <= fim:
        passos = passos + 1 # contou mais uma comparacao
        meio = (inicio + fim) // 2

        if lista[meio] == alvo:
            return meio, passos # achou!
        if lista[meio] < alvo:
            inicio = meio + 1 # o alvo esta na metade da DIREITA
        else:
            fim = meio - 1 # o alvo esta na metade da ESQUERDA

    return -1, passos # nao achou

# -----
# 3) Mede o tempo de uma busca
# time.perf_counter() devolve o "cronometro" do computador em segundos.
# -----
def medir_tempo(funcao_de_busca, lista, alvo):
    comeco = time.perf_counter()
    posicao, passos = funcao_de_busca(lista, alvo)
    fim = time.perf_counter()

    tempo_ms = (fim - comeco) * 1000 # segundos -> milissegundos
    return posicao, passos, tempo_ms
```

```

# -----
# 4) Programa principal
# -----

# Tudo o que for mostrado na tela tambem fica guardado aqui, para o PDF.
relatorio = []

def mostrar(texto=""):
    print(texto)
    relatorio.append(texto)

# A lista precisa estar ordenada por causa da busca binaria.
lista = list(range(TAMANHO))      # 0, 1, 2, 3, ... 999999
alvo = TAMANHO - 1                # o ULTIMO numero = pior caso para a linear

mostrar("COMPARACAO POR TEMPO: BUSCA LINEAR x BUSCA BINARIA")
mostrar("=" * 66)
mostrar(f"Tamanho da lista ... {TAMANHO} numeros (de 0 ate {TAMANHO - 1})")
mostrar(f"Numero procurado ... {alvo} (o ultimo da lista = pior caso)")
mostrar(f"Execucoes ..... {EXECUCOES}")
mostrar()

tempos_linear = []
tempos_binaria = []

for execucao in range(1, EXECUCOES + 1):
    mostrar(f"EXECUCAO {execucao}")

    # 1o resultado: busca linear
    posicao1, passos1, tempo1 = medir_tempo(busca_linear, lista, alvo)
    mostrar(f" 1o resultado - busca linear : posicao {posicao1} | "
           f"{passos1:>7} passos | {tempo1:>9.4f} ms")

    # 2o resultado: busca binaria
    posicao2, passos2, tempo2 = medir_tempo(busca_binaria, lista, alvo)
    mostrar(f" 2o resultado - busca binaria : posicao {posicao2} | "
           f"{passos2:>7} passos | {tempo2:>9.4f} ms")

    mostrar(f" --> a binaria foi {tempo1 / tempo2:.0f} vezes mais rapida")
    mostrar()

    tempos_linear.append(tempo1)
    tempos_binaria.append(tempo2)

# Media dos tempos das execucoes
media_linear = sum(tempos_linear) / EXECUCOES
media_binaria = sum(tempos_binaria) / EXECUCOES

mostrar(f"MEDIA DAS {EXECUCOES} EXECUCOES")
mostrar("=" * 66)
mostrar(f" Busca linear - O(n) ..... {media_linear:>9.4f} ms")
mostrar(f" Busca binaria - O(log n) .. {media_binaria:>9.4f} ms")
mostrar(f" A busca binaria foi {media_linear / media_binaria:.0f} vezes mais rapida.")
mostrar()
mostrar("PASSOS NO PIOR CASO (o que explica a diferenca de tempo)")
mostrar("=" * 66)
mostrar(f" Busca linear : {passos1} passos (olhou a lista toda)")
mostrar(f" Busca binaria : {passos2} passos (cortou a lista no meio {passos2} vezes)")

# -----
# 5) Salva os resultados em arquivos, para o gerar_pdf.py usar
# -----
(PASTA / "resultado.txt").write_text("\n".join(relatorio), encoding="utf-8")

linhas_csv = ["execucao,tempo_linear_ms,tempo_binaria_ms"]
for i in range(EXECUCOES):
    linhas_csv.append(f"{i + 1},{tempos_linear[i]:.4f},{tempos_binaria[i]:.4f}")
(PASTA / "tempos.csv").write_text("\n".join(linhas_csv), encoding="utf-8")

print()
print("Resultados salvos em: resultado.txt e tempos.csv")

```